INHERITANCE

- Inheritance is an **Object-Oriented Programming (OOP)** feature that allows a **child class** to acquire properties and methods from a **parent class**.
- It promotes **code reusability** and **better organization**.

Using Members of One Class inside Another Class:

We can use members of one class inside another class by using the following ways

1) By Composition (Has-A Relationship)

2) By Inheritance (IS-A Relationship)

1) By Composition (Has-A Relationship):

- By using Class Name or by creating object we can access members of one class inside another class is nothing but composition (Has-A Relationship).
- The main advantage of Has-A Relationship is Code Reusability.

Example:--

```python
class Engine:
    """ This is Engine class """
    a=10
    def __init__(self):
        print("I am Engine Constructor")
        self.b=20

    def m1(self):
        print("Engine Specific Functionality")


class Car:
    """ This is Car class """

    def __init__(self):
        print("I am Car Constructor")
        self.engine=Engine()

    def m2(self):
        print('Car using Engine Class Functionality')
        print(self.engine.a)
        print(self.engine.b)
        self.engine.m1()

c = Car()
c.m2()
```

Output

```
I am Car Constructor
I am Engine Constructor
Car using Engine Class Functionality
10
20
Engine Specific Functionality

=== Code Execution Successful ===
```

Example 2:

```
class Car:
    def __init__(self,name,model,color):
        self.name=name
        self.model=model
        self.color=color

    def get_car_info(self):
        print(f"Employee car name is :- {self.name}, Car Model is :- {self.model}, Car color is {self.color}.")

class Employee:
    def __init__(self,ename,eno,car):
        self.ename=ename
        self.eno=eno
        self.car=car

    def emp_all_info(self):
        print("Employee Name:",self.ename)
        print("Employee Number:",self.eno)
        print("Employee Car Info:")
        self.car.get_car_info()

c=Car("BMW","2026","BLACK")
e=Employee('Mukesh Ambani',1234567,c)
e.emp_all_info()
```

Output

```
Employee Name: Mukesh Ambani
Employee Number: 1234567
Employee Car Info:
Car name is :- BMW, Car Model is :- 2026, Car color is BLACK.

=== Code Execution Successful ===
```

Example 3:

```
class X:
    a = 10
    def __init__(self):
        self.b = 20
    def m1(self):
        print("m1 method of X class")

class Y:
    c = 30
    def __init__(self):
        self.d = 40
    def m2(self):
        print("m2 method of Y class")

    def m3(self):
        x1 = X()
        print(x1.a)      # Accessing static variable of X via instance
        print(x1.b)      # Accessing instance variable of X
        x1.m1()          # Calling method of X
        print(Y.c)       # Accessing static variable of Y via class name
        print(self.d)    # Accessing instance variable of Y
        self.m2()        # Calling method of Y
        print("m3 method of Y class")

y1 = Y()
y1.m3()
```

```
Output
10
20
m1 method of X class
30
40
m2 method of Y class
m3 method of Y class

=== Code Execution Successful ===
```

2) By Inheritance (IS-A Relationship):

What ever variables, methods and constructors available in the parent class by default available to the child classes and we are not required to rewrite.

Hence the main advantage of inheritance is Code Reusability and we can extend existing functionality with some more extra functionality.

Syntax: class childclass(parentclass)

```
class P:
    a=10
    def __init__(self):
        self.b=10

    def m1(self):
        print('Parent instance method')

    @classmethod
    def m2(cls):
        print('Parent class method')

    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    pass

c=C()
print(c.a)
print(c.b)
c.m1()
c.m2()
c.m3()
```

Output

```
10
10
Parent instance method
Parent class method
Parent static method

=== Code Execution Successful ===
```

```
class P:
   10 methods
class C(P):
   5 methods
```

In the above example Parent class contains 10 methods and these methods automatically available to the child class and we are not required to rewrite those methods (Code Reusability) Hence child class contains 15 methods.

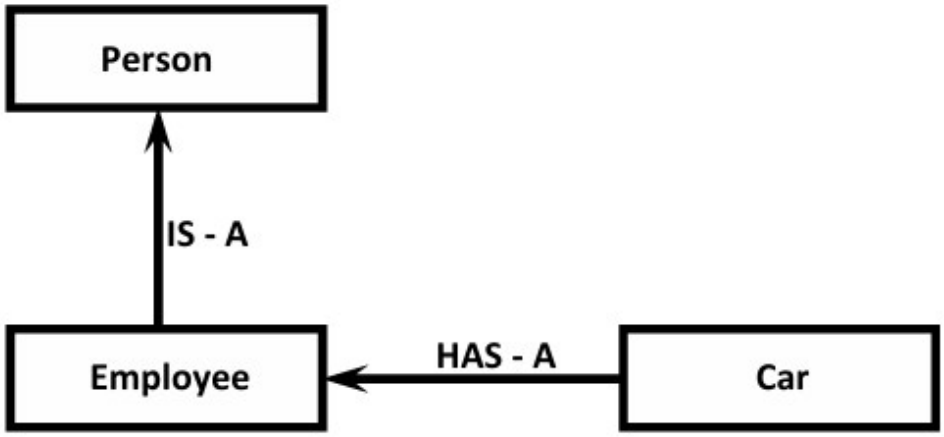
Demo program for Inheritance:

```
class Person:
   def __init__(self, name, age):
      self.name = name
      self.age = age

   def display_person(self):
      print('I am person instance method')

class Employee(Person):
   def __init__(self, name, age, eno, esal):
      # Calls the parent constructor to initialize name and age
      super().__init__(name, age)
      self.eno = eno
      self.esal = esal

   def display_employee(self):
      print("I am Employee instance method and Coding Python is very easy")

   def emp_info(self):
      print("Employee Name:", self.name)
      print("Employee Age:", self.age)
      print("Employee Number:", self.eno)
      print("Employee Salary:", self.esal)

# Creating an instance of Employee
e = Employee('Prateek', 48, 100, 10000)
e.display_person()   # Inherited from Person
e.display_employee()       # Defined in Employee
e.emp_info()     # Defined in Employee
```

```
Output
I am person instance method
I am Employee instance method and Coding Python is very easy
Employee Name: Prateek
Employee Age: 48
Employee Number: 100
Employee Salary: 10000

=== Code Execution Successful ===
```

IS-A vs HAS-A Relationship:

- **If we want to extend existing functionality with some more extra functionality then we should go for IS-A Relationship.**
- **If we dont want to extend and just we have to use existing functionality then we should go for HAS-A Relationship.**

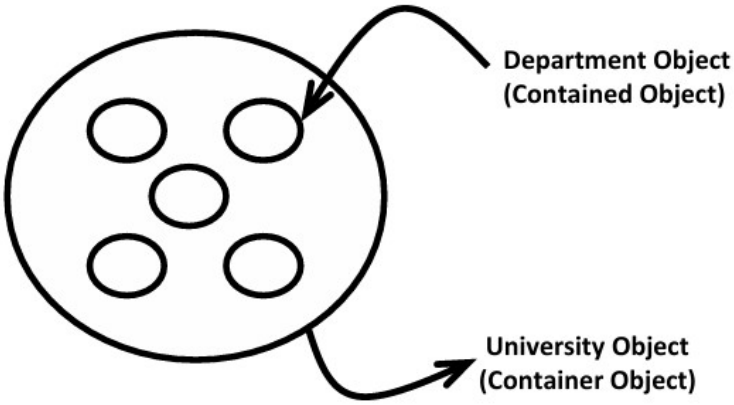


Composition vs Aggregation:

Composition:

Without existing container object if there is no chance of existing contained object then the container and contained objects are strongly associated and that strong association is nothing but Composition.

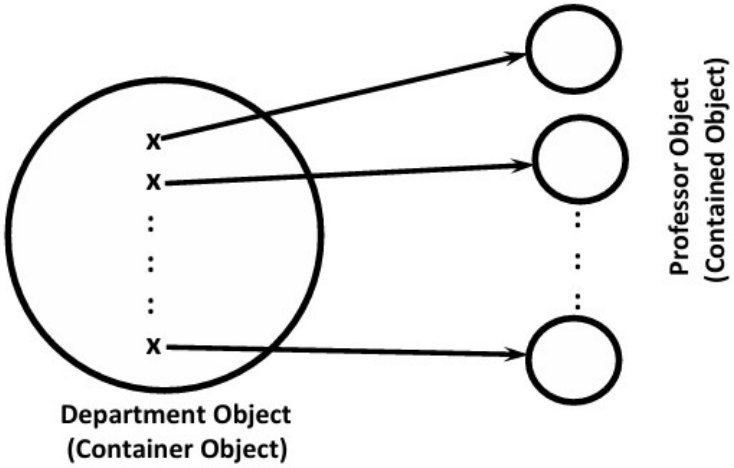
Eg: University contains several Departments and without existing university object there is no chance of existing Department object. Hence University and Department objects are strongly associated and this strong association is nothing but Composition.



Aggregation:

Without existing container object if there is a chance of existing contained object then the container and contained objects are weakly associated and that weak association is nothing but Aggregation.

Eg: Department contains several Professors. Without existing Department still there may be a chance of existing Professor. Hence Department and Professor Objects are weakly associated, which is nothing but Aggregation.



#Coding Example:

```
class Student:
   collegeName='ABC INSTITUTE OF TECHNOLOGY'

   def __init__(self,name):
      self.name=name

print(Student.collegeName)

s=Student('Prateek')
print(s.name)
```

In the above example without existing Student object there is no chance of existing his name. Hence Student Object and his name are strongly associated which is nothing but Composition.

But without existing Student object there may be a chance of existing collegeName. Hence Student object and collegeName are weakly associated which is nothing but Aggregation.

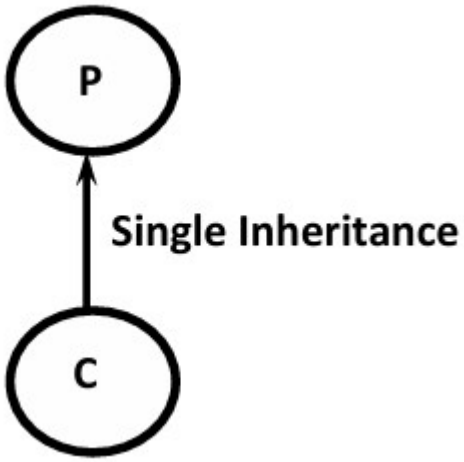
Conclusion:

The relation between object and its instance variables is always Composition where as the relation between object and static variables is Aggregation.

Types of Inheritance:

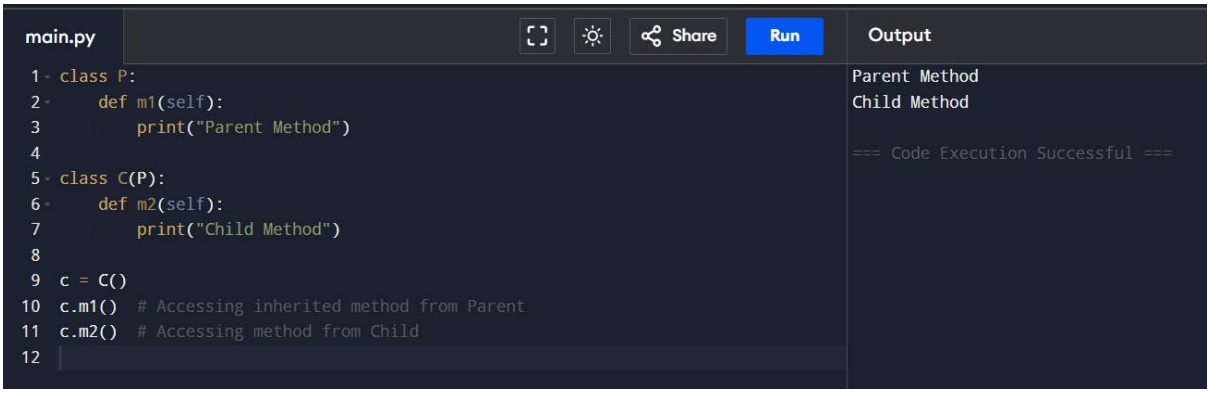
1) Single Inheritance:

The concept of inheriting the properties from one class to another class is known as single inheritance.



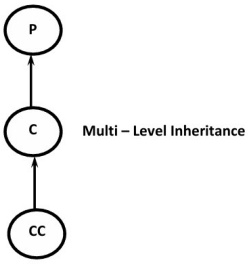
Example:

```
class P:
   def m1(self):
      print("Parent Method")

class C(P):
   def m2(self):
      print("Child Method")

c = C()
c.m1()  # Accessing inherited method from Parent
c.m2()  # Accessing method from Child
```



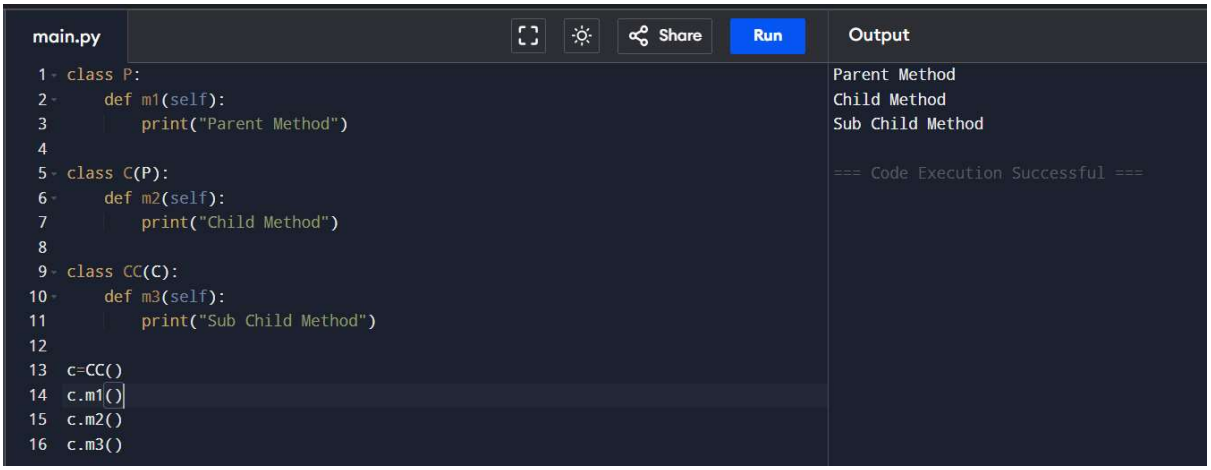
2) Multi Level Inheritance:

The concept of inheriting the properties from multiple classes to single class with the concept of one after another is known as multilevel inheritance.



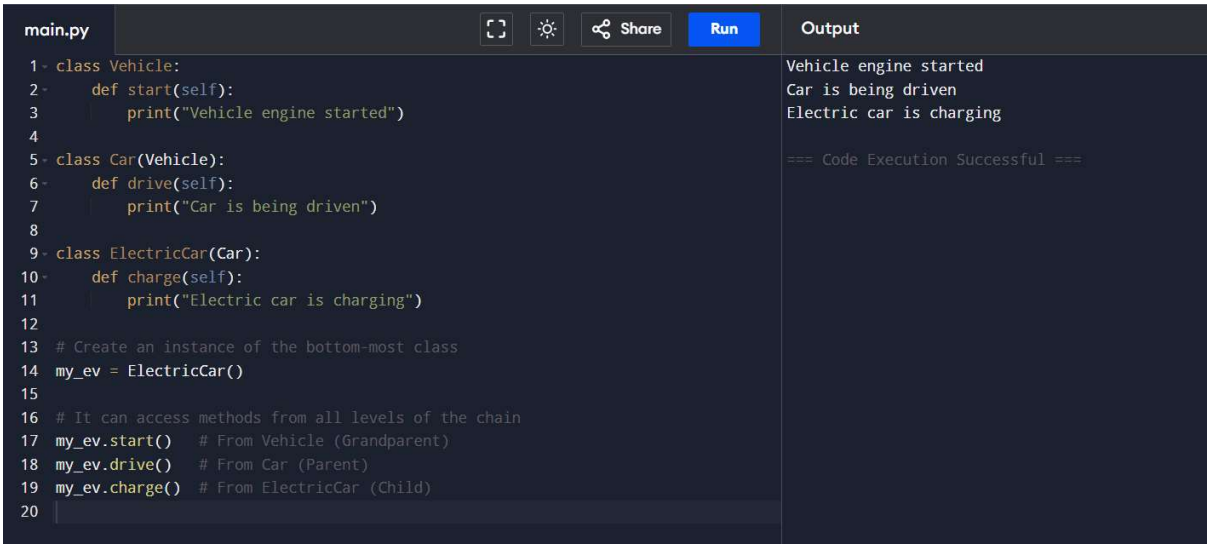
Example:

```
class P:
    def m1(self):
        print("Parent Method")

class C(P):
    def m2(self):
        print("Child Method")

class CC(C):
    def m3(self):
        print("Sub Child Method")

c=CC()
c.m1()
c.m2()
c.m3()
```



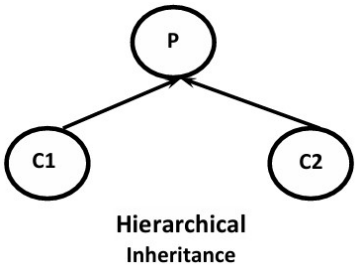
Example :

```python
class Vehicle:
    def start(self):
        print("Vehicle engine started")

class Car(Vehicle):
    def drive(self):
        print("Car is being driven")

class ElectricCar(Car):
    def charge(self):
        print("Electric car is charging")

# Create an instance of the bottom-most class
my_ev = ElectricCar()

# It can access methods from all levels of the chain
my_ev.start()   # From Vehicle (Grandparent)
my_ev.drive()   # From Car (Parent)
my_ev.charge()  # From ElectricCar (Child)
```



3) Hierarchical Inheritance:

The concept of inheriting properties from one class into multiple classes which are present at same level is known as Hierarchical Inheritance.




```
class P:
   def m1(self):
      print("Parent Method")

class C1(P):
   def m2(self):
      print("Child1 Method")

class C2(P):
   def m3(self):
      print("Child2 Method")

c1 = C1()
c1.m1()  # Inherited from P
c1.m2()  # Defined in C1

c2 = C2()
c2.m1()  # Inherited from P
c2.m3()  # Defined in C2
```

main.py

```python
class P:
    def m1(self):
        print("Parent Method")

class C1(P):
    def m2(self):
        print("Child1 Method")

class C2(P):
    def m3(self):
        print("Child2 Method")

c1 = C1()
c1.m1()  # Inherited from P
c1.m2()  # Defined in C1

c2 = C2()
c2.m1()  # Inherited from P
c2.m3()  # Defined in C2
```

Output

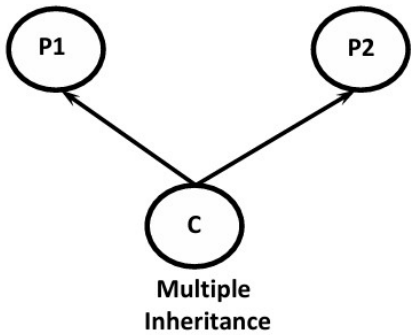
```
Parent Method
Child1 Method
Parent Method
Child2 Method

=== Code Execution Successful ===
```

Example:

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating.")

# Child Class 1
class Dog(Animal):
    def bark(self):
        print(f"{self.name} says: Woof!")

# Child Class 2
class Cat(Animal):
    def meow(self):
        print(f"{self.name} says: Meow!")

# Create objects of different child classes
d = Dog("Buddy")
c = Cat("Whiskers")

# Both access shared parent method
d.eat()
c.eat()

# Each accesses its own unique method
d.bark()
c.meow()
```

4) Multiple Inheritance:

The concept of inheriting the properties from multiple classes into a single class at a time, is known as multiple inheritance.



Example:

```
class P1:
   def m1(self):
      print("Parent1 Method")

class P2:
   def m2(self):
      print("Parent2 Method")

class C(P1, P2):
   def m3(self):
      print("Child Method")

c = C()
c.m1()  # Inherited from P1
c.m2()  # Inherited from P2
c.m3()  # Defined in C
```

main.py

```python
class P1:
    def m1(self):
        print("Parent1 Method")

class P2:
    def m2(self):
        print("Parent2 Method")

class C(P1, P2):
    def m3(self):
        print("Child Method")

c = C()
c.m1()  # Inherited from P1
c.m2()  # Inherited from P2
c.m3()  # Defined in C
```

Output

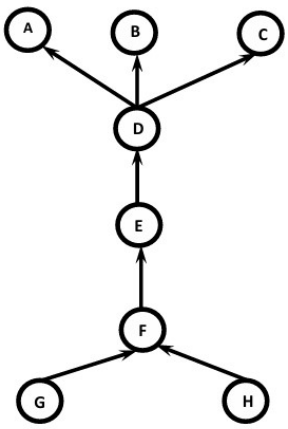
```
Parent1 Method
Parent2 Method
Child Method

=== Code Execution Successful ===
```

If the same method is inherited from both parent classes, then Python will always consider the order of Parent classes in the declaration of the child class.

class C(P1, P2): → P1 method will be considered
class C(P2, P1): → P2 method will be considered

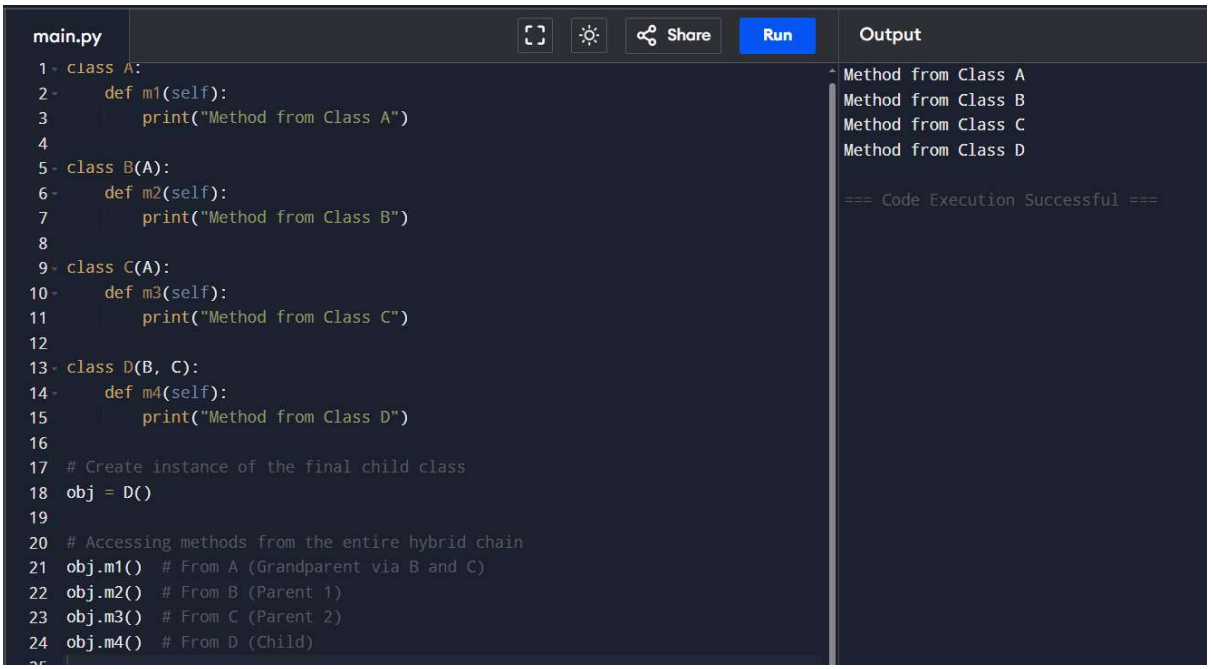
5) Hybrid Inheritance:

Combination of Single, Multi level, multiple and Hierarchical inheritance is known as Hybrid Inheritance.



Example:

```
class A:
    def m1(self):
        print("Method from Class A")

class B(A):
    def m2(self):
        print("Method from Class B")

class C(A):
    def m3(self):
        print("Method from Class C")

class D(B, C):
    def m4(self):
        print("Method from Class D")

# Create instance of the final child class
obj = D()

# Accessing methods from the entire hybrid chain
obj.m1()  # From A (Grandparent via B and C)
obj.m2()  # From B (Parent 1)
obj.m3()  # From C (Parent 2)
obj.m4()  # From D (Child)
```



Method Resolution Order (MRO):

- In Hybrid Inheritance the method resolution order is decided based on MRO algorithm.

• This algorithm is also known as C3 algorithm.
• Samuele Pedroni proposed this algorithm.
• It follows DLR (Depth First Left to Right) i.e Child will get more priority than Parent.
• Left Parent will get more priority than Right Parent.

Example:

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print(D.mro())
# Output: [D, B, C, A, object]

print(D.__mro__)
```

super() Method:

super() is a built-in method which is useful to call the super class constructors, variables and methods from the child class.

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, rollno, marks):
        super().__init__(name, age)
        self.rollno = rollno
        self.marks = marks

    def display(self):
        super().display()  # Calls display from Person
        print(f"Roll No: {self.rollno}")
        print(f"Marks: {self.marks}")

s1 = Student(Prateek’, 22, 101, 90)
s1.display()
```

main.py

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, rollno, marks):
        super().__init__(name, age)
        self.rollno = rollno
        self.marks = marks

    def display(self):
        super().display()  # Calls display from Person
        print(f"Roll No: {self.rollno}")
        print(f"Marks: {self.marks}")

s1 = Student('Prateek', 22, 101, 90)
s1.display()
```

Output

```
Name: Prateek
Age: 22
Roll No: 101
Marks: 90

=== Code Execution Successful ===
```

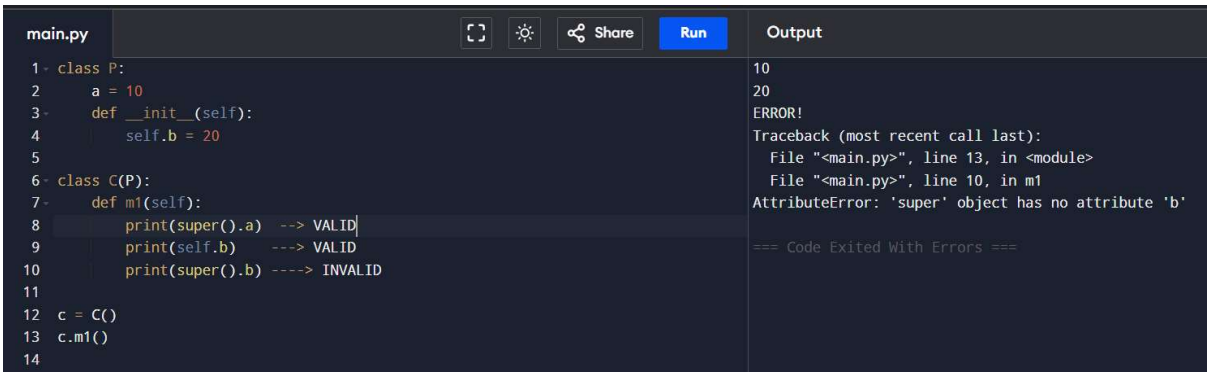
Example

```python
class P:
    a = 10
    def __init__(self):
        self.b = 10
    def m1(self):
        print('Parent instance method')
    @classmethod
    def m2(cls):
        print('Parent class method')
    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    a = 888
    def __init__(self):
        self.b = 999
        super().__init__()  # This resets self.b to 10
        print(super().a)    # Accesses class variable 'a' from P
        super().m1()        # Calls P's instance method
        super().m2()        # Calls P's class method
        super().m3()        # Calls P's static method

c = C()
print(c.a)
print(c.b)
```

Various Important Points about super():

1. From child class we are not allowed to access parent class instance variables by using super(), Compulsory we should use self only. But we can access parent class static variables by using super().

```
class P:
    a = 10
    def __init__(self):
        self.b = 20

class C(P):
    def m1(self):
        print(super().a)  --> VALID
        print(self.b)    ---> VALID
        print(super().b) ----> INVALID……ERROR WILL COME

c = C()
c.m1()
```



2. From child class constructor and instance method, we can access parent class instance method, static method and class method by using super()

```
class P:
    def __init__(self):
        print('Parent Constructor')

    def m1(self):
        print('Parent instance method')

    @classmethod
    def m2(cls):
        print('Parent class method')

    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    def __init__(self):
        print("--- Inside Child Constructor ---")
        super().__init__()
        super().m1()
        super().m2()
        super().m3()

    def m1(self):
        print("\n--- Inside Child m1 Method ---")
        super().__init__()
        super().m1()
        super().m2()
        super().m3()

c = C()
c.m1()
```

main.py

```python
class P:
    def __init__(self):
        print('Parent Constructor')

    def m1(self):
        print('Parent instance method')

    @classmethod
    def m2(cls):
        print('Parent class method')

    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    def __init__(self):
        print("--- Inside Child Constructor ---")
        super().__init__()
        super().m1()
        super().m2()
        super().m3()

    def m1(self):
        print("\n--- Inside Child m1 Method ---")
        super().__init__()
        super().m1()
        super().m2()
        super().m3()

c = C()
c.m1()
```

Output

```
--- Inside Child Constructor ---
Parent Constructor
Parent instance method
Parent class method
Parent static method

--- Inside Child m1 Method ---
Parent Constructor
Parent instance method
Parent class method
Parent static method

=== Code Execution Successful ===
```

3. From child class, class method we cannot access parent class instance methods and constructors by using super() directly(but indirectly possible). But we can access parent class static and class methods.

Example:

```
class P:
    def __init__(self):
        print('Parent Constructor')

    def m1(self):
        print('Parent instance method')

    @classmethod
    def m2(cls):
        print('Parent class method')

    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    @classmethod
    def m1(cls):
        #super().__init__()--->invalid
        #super().m1()--->invalid
        super().m2()
        super().m3()
c = C()
C.m1()
```

main.py

```python
class P:
    def __init__(self):
        print('Parent Constructor')

    def m1(self):
        print('Parent instance method')

    @classmethod
    def m2(cls):
        print('Parent class method')

    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    @classmethod
    def m1(cls):
        #super().__init__()--->invalid
        #super().m1()--->invalid
        super().m2()
        super().m3()
c = C()
C.m1()
```

Output

```
Parent Constructor
Parent class method
Parent static method

=== Code Execution Successful ===
```

Indirectly, From child class, class method we cannot access parent class instance methods and constructors by using super()

```python
class A:
    def __init__(self):
        print('Parent constructor')

    def m1(self):
        print('Parent instance method')

class B(A):
    @classmethod
    def m2(cls):
        super(B,cls).__init__(cls)
        super(B,cls).m1(cls)

B.m2()
```

4. In child class static method we are not allowed to use super() generally (But in special way we can use)

```
class P:
    def __init__(self):
        print('Parent Constructor')
    def m1(self):
        print('Parent instance method')
    @classmethod
    def m2(cls):
        print('Parent class method')
    @staticmethod
    def m3():
        print('Parent static method')

class C(P):
    @staticmethod
    def m1():
        super().m1()    -->invalid
        super().m2()    --->invalid
        super().m3()    --->invalid

c = C()
C.m1()
```

How to Call Parent Class Static Method from Child Class Static Method by using super():

```
1)  class A:
2)
3)      @staticmethod
4)      def m1():
5)          print('Parent static method')
6)
7)  class B(A):
8)      @staticmethod
9)      def m2():
10)         super(B,B).m1()
11)
12) B.m2()
```

Output: Parent static method

Interview question ?

Using multiple inheritance,

Print → (a+b)(a-b)

One parent have -> a+b

Other parent have -> a-b

Child to return -> (a+b)(a-b)

```
class Parent1:
    """ parent1 class """
    def __init__(self):
        print("I am parent 1 constructor")
    def get_sum_str(self):
        return "(a+b)"
class Parent2:
    """ parent2 class """
    def __init__(self):
        print("I am parent 2 constructor")
    def get_diff_str(self):
        return "(a-b)"

class Child(Parent1, Parent2):
    """ Child class """
    def __init__(self):
        print("I am Child constructor")

    def display_formula(self):
        # Combining methods from both parents
        part1 = self.get_sum_str()
        part2 = self.get_diff_str()

        print(f"The formula is: {part1}{part2}")

# Execution
obj = Child()
obj.display_formula()
```

main.py

```python
class Parent1:
    """ parent1 class """
    def __init__(self):
        print("I am parent 1 constructor")

    def get_sum_str(self):
        return "(a+b)"

class Parent2:
    """ parent2 class """
    def __init__(self):
        print("I am parent 2 constructor")

    def get_diff_str(self):
        return "(a-b)"

class Child(Parent1, Parent2):
    """ Child class """
    def __init__(self):
        print("I am Child constructor")

    def display_formula(self):
        # Combining methods from both parents
        part1 = self.get_sum_str()
        part2 = self.get_diff_str()

        print(f"The formula is: {part1}{part2}")

# Execution
obj = Child()
obj.display_formula()
```

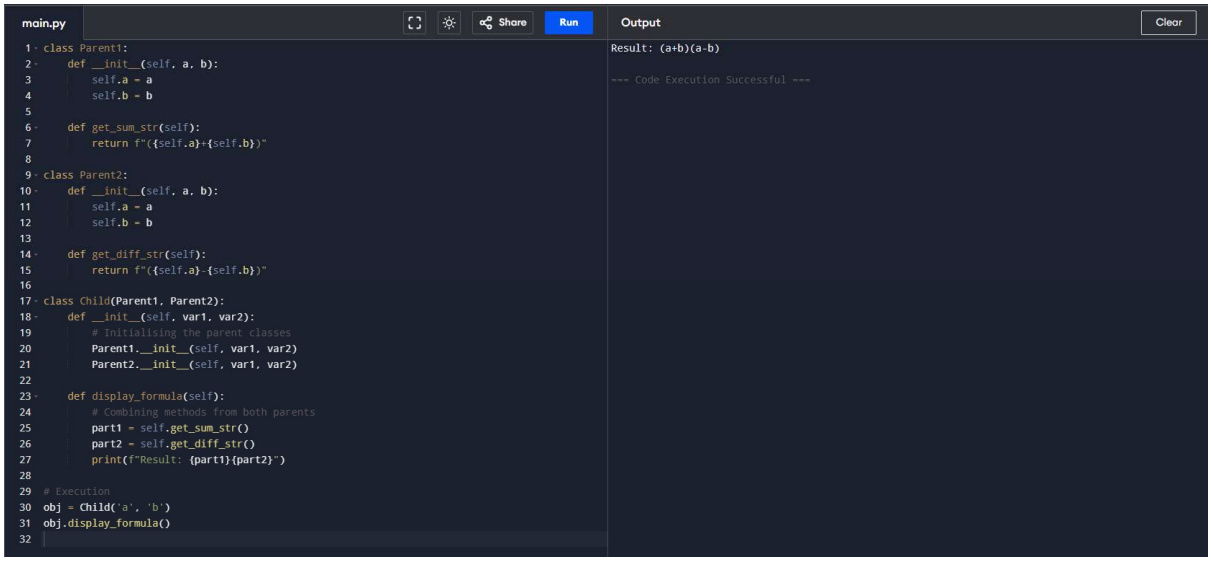
Output

```
I am Child constructor
The formula is: (a+b)(a-b)

=== Code Execution Successful ===
```

Provide Variable inside constructor then we have to write code like below

```python
class Parent1:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def get_sum_str(self):
        return f"({self.a}+{self.b})"
```

```
class Parent2:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def get_diff_str(self):
        return f"({self.a}-{self.b})"

class Child(Parent1, Parent2):
    def __init__(self, var1, var2):
        # Initialising the parent classes
        Parent1.__init__(self, var1, var2)
        Parent2.__init__(self, var1, var2)

    def display_formula(self):
        # Combining methods from both parents
        part1 = self.get_sum_str()
        part2 = self.get_diff_str()
        print(f"Result: {part1}{part2}")

# Execution
obj = Child('a', 'b')
obj.display_formula()
```

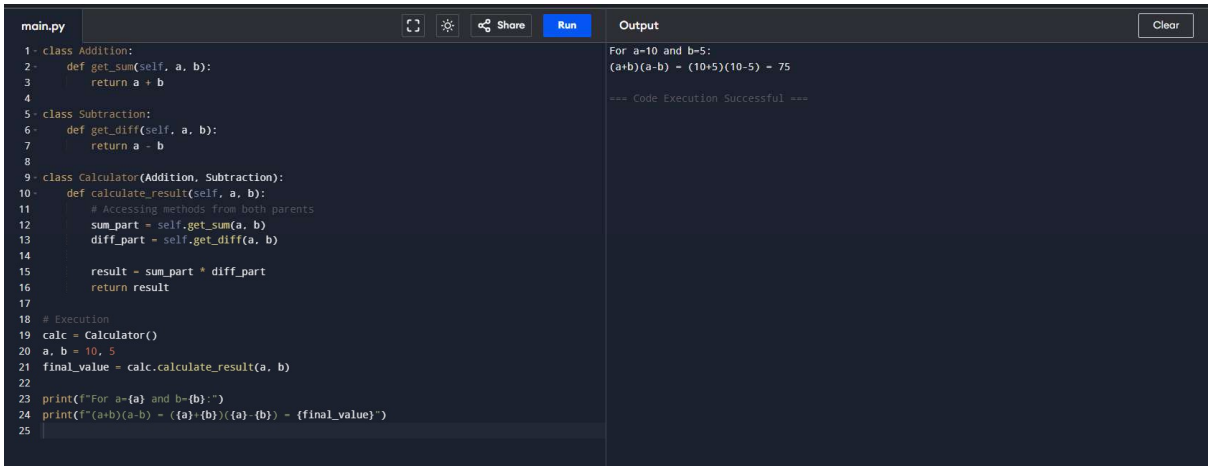


If they will ask like, they will provide value as

A = 10

B = 5

Then we have to write code like below

```
class Addition:
    def get_sum(self, a, b):
        return a + b


class Subtraction:
    def get_diff(self, a, b):
        return a - b
```

```python
class Calculator(Addition, Subtraction):
    def calculate_result(self, a, b):
        # Accessing methods from both parents
        sum_part = self.get_sum(a, b)
        diff_part = self.get_diff(a, b)

        result = sum_part * diff_part
        return result

# Execution
calc = Calculator()
a, b = 10, 5
final_value = calc.calculate_result(a, b)

print(f"For a={a} and b={b}:")
print(f"(a+b)(a-b) = ({a}+{b})({a}-{b}) = {final_value}")
```